## 本章路線

這章要回答的核心問題是：當工作集可能大於實體記憶體時，缺頁、替換與配置政策如何維持正確性並控制延遲？

**前置知識。** paging 位址轉換、page protection fault、儲存裝置延遲。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 學習目標

- 追蹤缺頁服務時序
- 比較 FIFO、OPT、LRU 與 clock
- 推導 page fault rate、frame allocation 與 thrashing

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.1 Background

- virtual memory 分開 logical address space、page-table mapping 與 physical residency。合法 virtual page 可以尚未載入；PTE 的 present/resident 位元只表示此架構定義的映射或駐留狀態，不能再把 `valid` 當成跨架構通用名稱。
- 處理 fault 時要分別檢查合法 virtual range、page 是否有 mapping、translation 是否 present/resident，以及 operation 是否符合 read/write/execute/user 等權限。合法位址沒有駐留可服務 page fault；非法範圍或權限不符必須拒絕。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.1 位址合法、內容來源與實體駐留各問一件事

固定4 KiB pages、兩個process A/B，兩者虛擬位址配置相同。表中frame名稱只在這個例子使用，沒有以相同virtual page number推論相同physical page。

| virtual範圍 | 合法性與權限 | 內容來源 | A目前映射 | B目前映射 |
|---|---|---|---|---|
| [0x0000,0x1000) | 合法、read/execute、不可write | 同一程式code | 共享F0 | 共享F0 |
| [0x1000,0x2000) | 合法、read/write、私有 | zero-fill-on-demand | 尚未resident | 尚未resident |
| [0x2000,0x3000) | 合法、read/write、私有 | 各自data | F1 | F2 |
| [0x3000,0x4000) | guard，無合法mapping | 無 | 無 | 無 |

每個process有3個合法virtual pages、2個resident mappings；全系統4個resident mappings卻只占3個不同frames，因code F0共享。把每個process的resident數直接相加會重複計共享frame。

A首次讀0x1004時，可配置清零頁後重啟；A寫0x0004則是權限違反；讀0x3004是guard存取，不能因有free frame就建立原本未授權的mapping。page table的nonpresent狀態必須結合VMA/backing metadata判斷，單看一個bit無法區分三種結果。

## 9.1 位址合法、內容來源與實體駐留各問一件事 （續）

virtual reservation、commit/backing承諾與resident是不同量。合法VMA可先保留位址；是否已承諾足夠RAM/swap由系統策略決定；resident只算目前在RAM的頁。因此「virtual空間大於RAM」本身不表示所有頁可同時寫入成功，也不表示記憶體永不配置失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.2 Demand paging

- demand paging 只在使用時載入頁面。page fault handler 驗證 fault 類型、找到空 frame或選 victim、讀入資料、更新 mapping 與 TLB，再重啟指令。
- 教材的單頁 trace 假設一次 fault 只服務被要求的 page。fault-around/read-ahead 可一次載入相鄰多頁；這會改變 prefetch 數量與 frame 壓力，但不能跳過每頁的合法性、權限與一致 mapping 檢查。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.2 Demand-paging fault：一次存取如何完成

固定 4 KiB pages、三個 frames 全部佔用；P5 位於合法 VMA 的 backing object，但尚未 resident。目前 frames 是 F0→P0、F1→dirty P1、F2→P2，替換政策選 F1。這是合法的 read fault，不是非法位址。

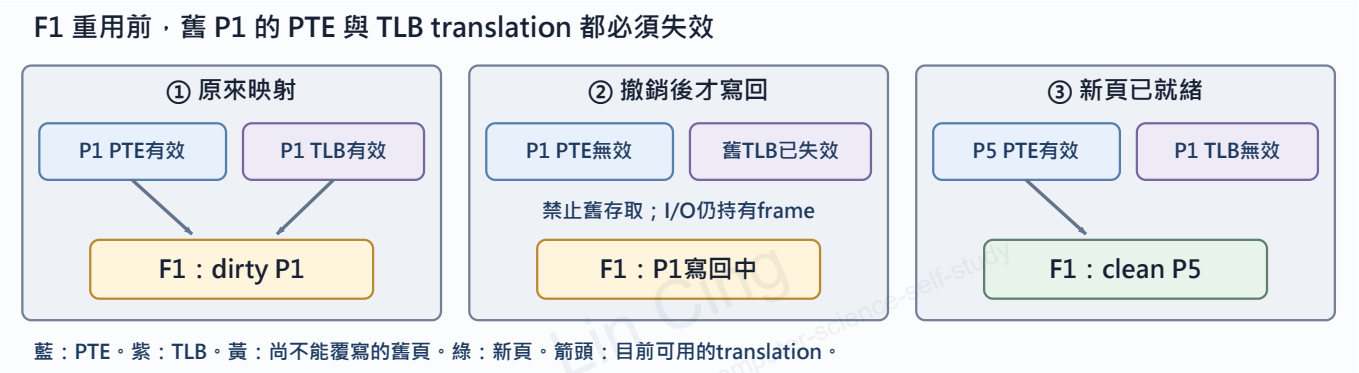
Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.2 Demand-paging fault：一次存取如何完成 （續）

| step | 可觀察狀態 | 操作 | 此時 P5 可否被讀 |
|---|---|---|---|
| 0 | P5 invalid；F0/F1/F2 已滿 | CPU 存取 P5，hardware trap | 否 |
| 1 | VMA 合法、backing object 存在、read 權限通過 | 分類為 nonresident fault | 否 |
| 2 | victim F1 dirty | 撤銷 P1 PTE，完成舊 TLB translation 的失效；禁止其他 CPU 再寫 F1 | 否 |
| 3 | F1 為 P1 writeback，仍由 fault/I/O 路徑持有 | 等 I/O 完成，不能覆蓋 F1 | 否 |
| 4 | F1 顯示 P5 loading | 從 backing store 讀入 4096 bytes | 否，mapping 未完成 |
| 5 | P5→F1；必要的 TLB entry 已失效 | 重啟 faulting instruction | 是 |

## 9.2 Demand-paging fault：一次存取如何完成 （續）




此模型計入 1 次 page fault、1 次 4096-byte disk write、1 次 4096-byte disk read，並有一個「撤銷舊 P1 翻譯」的 TLB 維護階段。單 CPU 模型只需本地失效；多 CPU 若曾快取該翻譯，須等待相關 shootdown 完成。安裝 P5 後是否另需失效或 barrier 由架構決定，不能把「所有架構恰好一條 TLB 指令」當結論。若 read 與 write 各 8 ms，且忽略其他開銷、不重疊，fault service 是 16 ms；clean victim 省去 writeback，為 8 ms。



## 9.2 Demand-paging fault：一次存取如何完成 （續）

有效存取時間需把 normal access 與 fault service 分開：

$$\mathrm{EAT} = (1-p)m + pF, \qquad p \le \frac{\mathrm{EAT}_{\max} - m}{F - m}.$$

令 m=200 ns、clean fault F=8,000,000 ns，要求 EAT≤400 ns 得 p≤200/7,999,800≈2.5001×10^-5；若每次 fault 必須先寫 dirty victim 再讀入，F=16,000,000 ns，門檻降為約1.2500×10^-5。fault rate 相同的應用，dirty-writeback 比例仍可令成本翻倍。

## 9.2　合法缺失、保護錯誤與 prefetch 的分流

fault handler 依序回答：

1. 位址是否落在合法 VMA 或 backing object？
2. 是否已有合法 mapping 的軟體描述，只是 PTE 目前 nonpresent？nonresident 不代表有有效的硬體 translation。
3. requested operation 是否符合 read/write/execute 權限？若只因 COW 暫時關閉 write bit，另走 COW 分支。
4. 是否有 free frame？沒有時選哪個 victim、是否 dirty/writeback/pinned？
5. 完成 I/O 後 mapping 與 TLB 是否一致，原指令可否安全重啟？

**重用過早的反例。** CPU0 清除 P1 PTE，CPU1 卻仍有 P1→F1 的 writable TLB entry。若 CPU0 立即把 P5 載入 F1，CPU1 對 P1 的 store 會改壞 P5。故「寫 PTE」與「舊 translation 已不可使用」不是同一事件。反向地，若 handler 完全不安裝 P5 PTE 就返回，原指令會再次 fault。正確不變量是重用 frame 前舊 mapping/translation 不可再存取它，重啟前新 mapping 的內容與權限已就緒。



## 9.2　合法缺失、保護錯誤與 prefetch 的分流 （續）

prefetch/read-ahead 可在一次 I/O 帶入 P5、P6；它減少後續 fault 次數，但 P6 仍必須合法、有 backing data、取得 frame 並安裝 mapping。不能因 batch read 成功就把非法相鄰頁自動加入 address space。

## 9.3 Copy-on-write

- fork 可讓 parent 與 child 暫時共享 read-only 實體頁，任一寫入才複製。這避免未寫頁的立即複製，但保留 page table 與 fault 管理成本。
- 共享狀態要追蹤引用數。最後引用消失後才回收 frame；寫入 fault 發生時，另一方可能仍在讀取。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.3 Copy-on-write：write fault 而非 missing-page fault

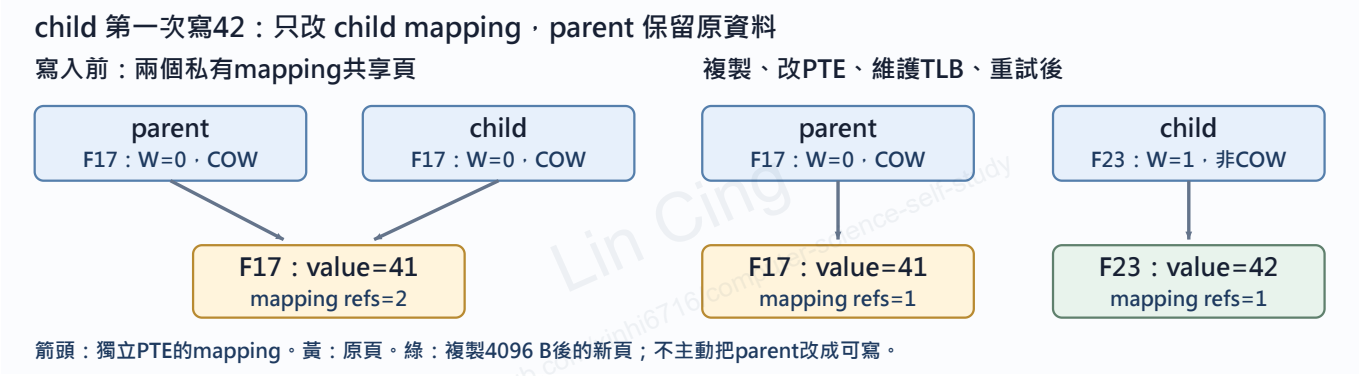
fork 後，parent 與 child 各自擁有 task 與 page-table metadata，但兩張 PTE 可暫時映射 F17；hardware 將兩者設為 read-only，OS 另記 COW。child 寫入時 translation 已存在，失敗的是 write permission，因此 fault handler 必須辨認 COW，而不能把它當成 backing-store page missing。

| 時刻 | event | parent PTE | child PTE | F17 | F23 |
|---|---|---|---|---|---|
| S0 | fork 完成 | F17, W=0, COW | F17, W=0, COW | value=41, refs=2 | 不存在 |
| S1 | child 寫入 | 不變 | 不變 | 41, refs=2 | 不存在 |
| S2 | 配置 F23，copy 4096 B，尚未公布新 PTE | F17 read-only | 仍指 F17 | 41, refs=2 | 41，由 handler 持有 |
| S3 | 公布 child PTE，完成翻譯維護，重試寫 42 | F17, W=0, COW | F23, W=1, COW=0 | 41, refs=1 | 42, refs=1, dirty |

Lin Cing

## 9.3 Copy-on-write：write fault 而非 missing-page fault （續）

必要順序是：驗證 COW fault→分配 frame→複製舊內容→更新 child PTE/TLB→重試 write。只清除 parent 的 COW 或把兩邊都改成 writable，會讓 child 直接改到 parent 的 F17。






## 9.3 Copy-on-write：write fault 而非 missing-page fault （續）

這裡 refs 只計入使用者 mapping，不把 handler 的暫時保留混入同一欄。child 的 fault 不必主動走訪並更新 parent PTE；parent 可以繼續保持唯讀。其後 parent 首次 write 再 fault 時，若確認 F17 是可獨佔的私有 anonymous page、沒有其他共享或引用限制，可只提升權限而不複製。僅憑「mapping 數等於 1」不能把 file cache 的共享頁任意改成私有可寫。若 fork 後立刻 exec，未被寫過的 COW 頁可直接解除映射，省下無用複製。

zero-fill-on-demand 的私有資料頁在被寫入前必須先清零，不能把舊 frame 的殘留資料暴露給另一 process。COW、zero-fill 與 demand paging 都會在 fault path 配置 frame，但觸發原因、資料來源與安全檢查不同。

## 9.4 Page replacement

- 記憶體不足時要選 victim。FIFO 以載入時間，LRU 以最近未使用，OPT 以未來最久才使用；OPT 提供離線下界而非可實作政策。
- Belady anomaly 說明增加 frames 不保證所有 replacement policy 的 page faults 單調下降；FIFO 可出現反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



### 9.4.1 LRU 近似與 Clock

- 精確 LRU 每存取需更新排序，硬體與併發成本高。clock 演算法用 reference bit 掃描，遇到 1 時清除並前進，遇到 0 時選擇 victim。
- 幾輪掃描後可能把活躍頁變得不活躍；掃描順序與停止條件定義實際 policy。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

### 9.4.2 Page fault 指標

- effective access time 可寫成 `(1-p)×memory_time + p×fault_service_time`。fault service 本身含保存、排程、I/O 與恢復。
- 若 memory=100 ns、fault=8 ms、要求 EAT<=200 ns，則 p <= 100/7,999,900，約 1.25×10^-5；設定基準不同仍使用同一個不等式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



### 9.4 四種替換策略的完整狀態差異

固定 frames=3，reference string 為 `1,2,3,4,1,2,5,1,2,3,4,5`。每一 row 是一次 page access，`H/F` 表示 hit/fault；resident 的次序是該政策的 metadata，不把它誤當成唯一正確物理 frame 編號。

| step/page | FIFO | LRU | OPT | CLOCK |
|---|---|---|---|---|
| 1/1 | [1]F | [1]F | [1]F | [1]F |
| 2/2 | [1,2]F | [1,2]F | [1,2]F | [1,2]F |
| 3/3 | [1,2,3]F | [1,2,3]F | [1,2,3]F | [1,2,3]F |
| 4/4 | [2,3,4]F | [2,3,4]F | [1,2,4]F | [4,2,3]F |
| 5/1 | [3,4,1]F | [3,4,1]F | [1,2,4]H | [4,1,3]F |
| 6/2 | [4,1,2]F | [4,1,2]F | [1,2,4]H | [4,1,2]F |
| 7/5 | [1,2,5]F | [1,2,5]F | [1,2,5]F | [5,1,2]F |
| 8/1 | [1,2,5]H | [2,5,1]H | [1,2,5]H | [5,1,2]H |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.4 四種替換策略的完整狀態差異 （續）

| step/page | FIFO | LRU | OPT | CLOCK |
|---|---|---|---|---|
| 9/2 | [1,2,5]H | [5,1,2]H | [1,2,5]H | [5,1,2]H |
| 10/3 | [2,5,3]F | [1,2,3]F | [2,5,3]F | [5,3,2]F |
| 11/4 | [5,3,4]F | [2,3,4]F | [5,3,4]F | [5,3,4]F |
| 12/5 | [5,3,4]H | [3,4,5]F | [5,3,4]H | [5,3,4]H |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.4 四種替換策略的完整狀態差異 （續）

總結是 FIFO=9、LRU=10、OPT=7、CLOCK=9 faults。victim sequence：

| policy | victim sequence（- 表示 free frame 或 hit） |
|---|---|
| FIFO | `-, -, -, 1, 2, 3, 4, -, -, 1, 2, -` |
| LRU | `-, -, -, 1, 2, 3, 4, -, -, 5, 1, 2` |
| OPT | `-, -, -, 3, -, -, 4, -, -, 1, 2, -` |
| CLOCK | `-, -, -, 1, 2, 3, 4, -, -, 1, 2, -` |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

FIFO 只看載入時間，所以 hit 不改變 queue；LRU 在 hit 時把頁移到最近端；OPT 知道未來，選擇下次使用最晚或不再使用的頁；clock 以 circular queue 與 reference bit 清除第一輪 chance。這些 metadata 是不可互換的。

**Belady anomaly**。同一 reference string 上，FIFO 在 3 frames 為 9 faults，在 4 frames 反而為 10；增加 frame 不保證每一種 policy 的 fault count 單調下降。LRU 與 OPT 屬 stack algorithms，在這個正式模型下有 monotonic nesting；實作近似不等於精確 LRU。

## 9.4　OPT 與近似 LRU 的可用性

OPT 需要完整未來 reference，只能作離線比較下界。它證明的是一個 reference string 的最低 eviction 次數，不是線上系統能預知應用程式。

硬體 reference bit 可重建最近使用資訊：每個取樣 period 把 bit 移入歷史 byte，數值較小者通常較久未使用；clock 只保留一位 reference，遇到1時清為0並給第二次機會。改變 clock hand、period 或清除時機，就會形成不同近似策略，不能只寫「clock=LRU」。

本例 CLOCK 的 hand 初始指 slot0；每次 hit 設 R=1、不移 hand；有 free slot 時依0、1、2填入且設 R=1；滿後由 hand 逐格掃，R=1先清0，遇R=0替換、設新頁R=1並將hand前移。第4次 reference4 先把三個R清零，再換 slot0 的1；第7次 reference5 再清一圈，換 slot0 的4。這些初始化規則是重建表格所需的完整政策。

有效存取時間只以 fault count 代入仍不完整，因 dirty victim 多一次 write transfer：

$$T_{\text{fault}} = T_{\text{trap}} + T_{\text{swapout(dirty)}} + T_{\text{swapin}}.$$

同一 fault rate 下，若一半 victim dirty，平均 fault service 會介於單一 read 與 write+read 路徑，必須保存 clean/dirty 比例。OPT 在此 cost model 也不一定和最低 fault count 完全同序。



## 9.4　LRU 為何不會出現 Belady anomaly

把所有曾存取的頁按最後存取時間從新到舊排成同一個 **recency stack**。每次存取頁 $x$，從原位置移走 $x$，再放到最前端；尚未出現的頁不在 stack。此順序的更新與 frame 數無關。

**歸納不變量。** 具有 $k$ frames 的精確 LRU，其 resident set 恰為 stack 前 $k$ 項，或不足 $k$ 時的全部項目。初始都空；hit 移到前端不改前 $k$ 集合；miss 把 $x$ 放首位，若已滿則原第 $k$ 項被擠出，正是 LRU victim。因此每次更新仍成立。

所以 $R_k(t) \subseteq R_{k+1}(t)$。若 $k+1$ frames 在時刻 $t$ fault，請求頁不在 $R_{k+1}(t-1)$，也不在其子集 $R_k(t-1)$，故 $k$ frames 同樣 fault。對時刻加總得 $F_{k+1} \le F_k$。這是一般證明，不能由本章單一12項 trace 代替。

FIFO 的載入順序受先前 fault 影響，而不同容量產生不同 fault；沒有同一條容量無關的 recency stack。CLOCK 只近似近期性，也不能直接套用上述包含證明。

## 9.4　OPT 的貪婪選擇與計費模型

固定離線 reference string、相同初始 resident set、每次 miss 成本1、沒有 dirty 寫回差別。一次滿 frame 的 fault 上，OPT 移除「下次被要求最晚」的頁 $q$，永不再用視為 $+\infty$。

**交換引理。** 假設某最優排程第一次在此處不同，移除 (p)。修改它，先移除 $q$、保留 (p)。在下一次要求 (p) 之前，不會先要求 $q$，因 $q$ 的下次使用不早於 (p)。故兩個排程可維持除 (p,q) 外相同的頁；原排程若先主動移除 $q$，差異可立即消除。否則到 (p) 被要求時，修改後排程已有 (p)，省下原排程的一次 fault；可用這次省下的傳輸載入 $q$ 並移除原排程此次移除的頁，讓 resident set 對齊，總傳輸不增加。若兩頁都不再用，則差異不影響成本。

上述對齊使用一次可提前的讀入；把任何提前讀入延至首次要求該頁，成本不增，也得到合法 demand-paging 排程。因此存在與 OPT 此步一致的最優排程。逐次消除第一個分歧，得到 OPT 最優。此證明的成本是等權 page-in 次數；不能直接推論有不同 page sizes、非均勻 I/O 或 dirty page-out 時仍最優。

額外 frame 的最優排程可模擬少一個 frame 的排程、暫不使用多出的空間，所以 OPT 最小成本也不隨容量增加而上升。這個模擬證明不要求不同 tie-breaking 的 OPT 實作逐步具有相同的 resident nesting。



## 9.4　LFU、MFU 與歷史頻率失效

頻率政策還須定義計數生命週期與同分規則。本例從空的3 frames 開始，載入後 count=1，hit 加1，eviction 即丟棄該頁計數，同分時選最早載入者。

輸入為 `1,1,1,2,3,4`。前五次後 resident={1,2,3}，counts={1:3,2:1,3:1}。第六次要求4時，LFU（least frequently used）移除2，結果{1:3,3:1,4:1}；MFU（most frequently used）移除1，結果{2:1,3:1,4:1}。二者並非 LRU 或 OPT 的別名。

LFU 的3次舊命中會讓1長時間留下，即使 locality 已轉移到{2,3,4}；可定期衰減計數以減輕舊歷史，但衰減週期本身又是一個政策參數。MFU 假設低頻頁可能剛載入、未來還有使用機會，這是工作負載假設，並無普遍最優保證。

## 9.4 CLOCK每一步的hand與R bits

此表把先前省略的實體slot、R bits與下一次掃描起點一併列出；hit只設對應R=1，不移hand。初始裝入free slots也不移hand，故前三次後hand仍0。

| t | page | slots0/1/2 | R0/R1/R2 | 下一個hand | H/F；victim |
|---|---|---|---|---|---|
| 1 | 1 | [1] | [1] | 0 | F；— |
| 2 | 2 | [1, 2] | [1, 1] | 0 | F；— |
| 3 | 3 | [1, 2, 3] | [1, 1, 1] | 0 | F；— |
| 4 | 4 | [4, 2, 3] | [1, 0, 0] | 1 | F；1 |
| 5 | 1 | [4, 1, 3] | [1, 1, 0] | 2 | F；2 |
| 6 | 2 | [4, 1, 2] | [1, 1, 1] | 0 | F；3 |
| 7 | 5 | [5, 1, 2] | [1, 0, 0] | 1 | F；4 |
| 8 | 1 | [5, 1, 2] | [1, 1, 0] | 1 | H；— |



## 9.4 CLOCK每一步的hand與R bits (續)

| t | page | slots0/1/2 | R0/R1/R2 | 下一個hand | H/F；victim |
|---|---|---|---|---|---|
| 9 | 2 | [5, 1, 2] | [1, 1, 1] | 1 | H；— |
| 10 | 3 | [5, 3, 2] | [0, 1, 0] | 2 | F；1 |
| 11 | 4 | [5, 3, 4] | [0, 1, 1] | 0 | F；2 |
| 12 | 5 | [5, 3, 4] | [1, 1, 1] | 0 | H；— |

## 9.4 FIFO容量增加卻多fault的完整比較

FIFO的queue次序由各容量自己的fault歷史決定。第7步後3-frame set含1，4-frame set卻不含1，故第8步大容量反而fault；不具有LRU的包含性。

| step/page | 3-frame queue | 4-frame queue | 累計faults 3/4 |
|---|---|---|---|
| 1/1 | [1]F | [1]F | 1/1 |
| 2/2 | [1,2]F | [1,2]F | 2/2 |
| 3/3 | [1,2,3]F | [1,2,3]F | 3/3 |
| 4/4 | [2,3,4]F | [1,2,3,4]F | 4/4 |
| 5/1 | [3,4,1]F | [1,2,3,4]H | 5/4 |
| 6/2 | [4,1,2]F | [1,2,3,4]H | 6/4 |
| 7/5 | [1,2,5]F | [2,3,4,5]F | 7/5 |
| 8/1 | [1,2,5]H | [3,4,5,1]F | 7/6 |



## 9.4 FIFO容量增加卻多fault的完整比較 （續）

| step/page | 3-frame queue | 4-frame queue | 累計faults 3/4 |
|---|---|---|---|
| 9/2 | [1,2,5]H | [4,5,1,2]F | 7/7 |
| 10/3 | [2,5,3]F | [5,1,2,3]F | 8/8 |
| 11/4 | [5,3,4]F | [1,2,3,4]F | 9/9 |
| 12/5 | [5,3,4]H | [2,3,4,5]F | 9/10 |

## 9.5 Frame allocation

- allocation policy 決定每個 process 能取得多少 frames。equal allocation 公平但不考慮需求；proportional allocation 依 size 或優先權配置。
- local replacement 只看自身 frames，global replacement 可從全體選 victim。global 提高短期利用率，但可能讓一個 process 的 fault 影響其他 process。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.5 Frame allocation：equal、proportional 與 rounding

equal allocation 若 total frames=F、process 數=m，先給每個 process $\lfloor F/m \rfloor$，餘數的分配規則必須明定。proportional allocation 依 process 大小或權重：

$$a_i = \left\lfloor F \frac{s_i}{\sum_j s_j} \right\rfloor,$$

再以最大餘數、priority 或固定順序分配剩下 frames。

固定 F=64，process sizes A=100、B=200、C=300，總大小600：

| process | raw share | floor | remainder | final |
|---|---|---|---|---|
| A | 10.667 | 10 | 0.667 | 11 |
| B | 21.333 | 21 | 0.333 | 21 |
| C | 32.000 | 32 | 0.000 | 32 |

floor 合計63，剩1 frame 依最大餘數給 A，得到 `11/21/32` 且總和64。若不保存 rounding 規則，同一個「proportional」百分比可產生不同配置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.5 Frame allocation：equal、proportional 與 rounding （續）

任何 process 的 quota 仍要有 correctness 下界：執行中 instruction、page-table walk 與 interrupt/fault 路徑所需 pages 不能被任意壓到零。fault handler 核心記憶體是系統成本，不應被直接算成每個 process 的負擔。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.5 Local 與 global replacement 的同事件分歧

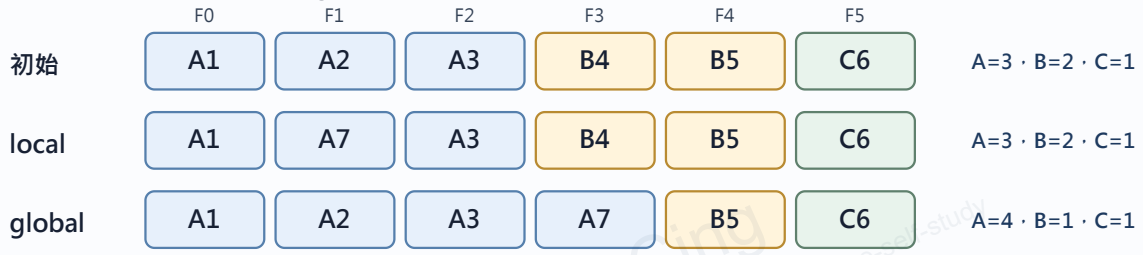
初始 resident sets 是 A={1,2,3}、B={4,5}、C={6}，共6 frames。last-use 時戳是 A1=10、A2=5、A3=8、B4=2、B5=7、C6=9；A 在沒有 free frame 時 fault page7。

| policy | victim 範圍 | victim | 結果 |
|---|---|---|---|
| local | 只在 A 的 frames | A2（A 內最久未用） | A={1,7,3}；B/C 不變 |
| global | 所有 process 的 frames | B4（全域最久未用） | A={1,2,3,7}；B={5}；C={6} |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.5 Local 與 global replacement 的同事件分歧 (續)

同一次 A7 fault：local / global 只重用一個 frame




三列都恰為6 frames；global 的下一次 B4 存取會 fault。

global 的 A fault 搶走 B4 的 frame，A 的 resident 數由3升4，B 由2降1，總數仍為6；既沒有刪掉 A3，也沒有憑空載入 B8。B 下一次讀4便再 fault，可能把局部壓力擴散成跨 process paging storm。local 保留隔離，但 A 無法借用其他 process 暫時閒置的 frame。



## 9.5 Local 與 global replacement 的同事件分歧 (續)

allocation 與 replacement 是兩個決策：allocation 決定每個 process 的 quota/上下界，replacement 決定 fault 時 victim pool。把二者寫成「global 就是沒有上限」會遺漏 residency set 與 admission control。

## 9.5 最小frame數不是working-set大小

架構允許一條指令同時碰多少code、source與destination pages，及fault後如何重啟，會決定執行所需的最小user frames。若指令不可部分完成，載入最後一個operand時卻逐出同指令必需的前一頁，便可能反覆fault而無法完成。這是correctness/progress下界；WSS則是所選時間窗的效能需求，兩者不可當同一常數。

本章proportional的11/21/32例子未加入額外minimum約束。若每個process另有下界 $l_i$，必須先查 $\sum_i l_i \le F$。可先給下界，再將餘額 $F - \sum l_i$ 依權重與明定rounding分配；直接算比例後各自取 `max(share,l_i)`，總和可能超過F。

例如F=6，三個process均需至少3個不可共享frames，總下界9>6；任何公平權重或replacement算法都無法同時滿足三者。必須暫停至少一個process、改變執行策略或增加frames。page-table/fault-handler的核心記憶體與裝置pin預留另計，不能把F的全部容量都先分給user quota。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.6 Thrashing

- thrashing 是系統花大部分時間搬頁，真正計算不足。當 allocated frames 小於 working-set 需求，fault rate 上升並形成惡性循環。
- working-set window 擷取近期引用頁。若 Σ working_set > available frames，系統需降低多程式度或增加記憶體。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.6 Working set、頁面需求與 thrashing

working-set window 固定為最近4次 references。三個 process 的固定尾部 references：

| process | recent references | working set | $WSS_i$ |
|---|---|---|---|
| A | 1,2,3,1,2,4,5,1 | {1,2,4,5} | 4 |
| B | 6,7,6,8,7 | {6,7,8} | 3 |
| C | 9,9,9 | {9} | 1 |

這裡A/B/C的頁身分互不共享，總需求 $D = 4 + 3 + 1 = 8$。可用 frames m=6，因此 D>m。若這8頁都仍是同一視窗內 active working set，系統沒有足夠 frames 同時維持全部 locality，工作集模型預測 thrashing 風險。若有共享frames，實體需求應計unique backing pages的union，不能無條件把各WSS相加。

反例邊界：短窗口可能把仍會使用的頁排除；長窗口把低頻頁算入 WSS；process 也可能因鎖、DMA 或單一慢 I/O 停住，而 CPU 低與 paging 無關。D>m 是壓力預測，仍需 fault rate、CPU utilization 與 I/O queue 的 runtime evidence。



## 9.6 PFF controller 與 hysteresis

page-fault frequency（PFF）直接用固定觀察窗的每頁存取 fault ratio。設定 upper=0.4、lower=0.1：

| window | 最近5次 F/H | PFF | 決策 |
|---|---|---|---|
| 過載 | F,F,F,F,F | 1.0 | 若可取得 frame，分配給該 process |
| 分配後 | F,H,H,H,H | 0.2 | 介於上下界，保留 |
| 持續局部命中 | H,H,H,H,H | 0.0 | 有記憶體壓力時可考慮回收一 frame |

兩個門檻形成 hysteresis：只有 high 才加 frame，只有低於 low 才減 frame，避免在單一門檻附近反覆搬頁。若高 PFF 但沒有 free frame，系統應降低多程式度、換出一個 process 或以 admission control 拒絕新工作；繼續提高多程式度只會把其他 process 推入 thrashing。

## 9.6 PFF controller 與 hysteresis （續）

PFF 控制器有一條可重建偽碼：

```
for each process every sample_window:
    if accesses_in_window == 0:
        continue
    rate = faults_in_window / accesses_in_window
    if rate > upper and free_frames > 0:
        grant_one_frame(process)
    else if rate > upper and free_frames == 0:
        choose one process to suspend
    else if rate < lower and memory_pressure and quota > minimum_required:
        reclaim_one_spare_frame(process)
```

window 太短會受 burst 雜訊影響，太長則反應慢；因此 threshold、window、suspension policy 必須一起報告。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.7 Memory-mapped files

- MAP_SHARED讓映射到同一file page的process共享更新，但應用的讀寫次序仍須相應記憶體同步。MAP_PRIVATE的修改進入私有COW頁，不寫回原檔。
- mmap建立mapping不代表data已resident，也不代表store已持久；page fault、visibility、writeback與durability是不同事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.7 Memory-mapped file：mapping、dirty 與 durability

file page 先進入 page cache，再由兩個 process 的 PTE 映射到相同 physical frame。shared mapping 的 ordinary load/store 操作 cache page；private mapping則在 write 時以 COW 建立私有 frame。頁面 cache hit 只表示不用讀 disk，不代表資料已持久。

以下是兩個**各自從 value=7 開始**的實驗，沒有把 shared write 與 private mapping 混成同一保證。

| shared 實驗 | P1 mapping | P2 mapping | cache F40 | backing file |
|---|---|---|---|---|
| S0 初始 | F40 | F40 | 7，clean | 7 |
| S1 P1 寫8，並用同步通知 P2 | F40 | F40 | 8，dirty | 7 |
| S2 P2 在同步後讀取 | F40 | F40 | 讀8；尚未持久 | 7 |
| S3 寫回完成且滿足指定 flush 契約 | F40 | F40 | 8，clean | 8 |



## 9.7 Memory-mapped file：mapping、dirty 與 durability （續）

| private 實驗 | P1 mapping/value | P2 mapping/value | page cache | backing file |
|---|---|---|---|---|
| V0 初始 | F40/7，COW | F40/7，COW | F40/7 | 7 |
| V1 只有 P2 寫9 | F40/7，未複製 | 新 F53/9 | F40/7 | 7 |
| V2 P1 再寫8 | 新 F52/8 | F53/9 | F40/7 | 7 |

V1 只複製4096 bytes，V2 累計8192 bytes。不能在 P2 單獨寫入時，把尚未寫入的 P1 也列成新私有 frame。Linux `MAP_PRIVATE` 不保證 mapping 建立後檔案的外部變更是否可見；本實驗明定沒有其他檔案寫入，故不把 private mapping 說成自動建立快照。

shared mapping 的跨 thread visibility 仍要符合記憶體同步；一般 ordinary write 與讀取之間不能只因有 mmap 就自動形成任意順序。`msync` 或其平台對應 API 觸發 writeback 與必要的ordering，但其 durability contract 仍依 API、文件系統、儲存 cache 與 flush 規則。

## 9.7 Memory-mapped file：mapping、dirty 與 durability （續）

private map 的 COW page 修改不應寫回原 file。頁面被 reclaim 時，dirty shared/file page 必須先安全寫回；dirty private file-backed COW page 寫到 swap 或 backing store，而不是覆蓋原始 file。writeback 失敗要保留 errored state，不能把頁標成 clean 後重用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.7 mmap 與 read/write 路徑的相同與差異

`read(fd,buf,n)` 需要 system call、copy data 到 user buffer，並更新 file offset；mmap 首次存取靠 page fault 建立 file page→PTE mapping，之後 load/store 由 MMU 完成。兩條路徑都可受益於 page cache，但 fault、copy、TLB 與 page-dirty tracking 的成本不同。

固定冷 cache、4 KiB page：

| path | system calls | data copies | TLB events | durability after write |
|---|---|---|---|---|
| read一次 | 1 | cache→user 4096 B | normal | read 不承諾 |
| write一次 | 1 | user→cache 4096 B | normal | 需 fsync/writeback/commit 才具指定持久性 |
| mmap首次read | 先前1次 mmap；此次為 fault/trap | 0次 cache→user copy；磁碟仍讀4096 B至cache | install/必要維護 | 不涉及寫入持久性 |
| mmap shared write | 0 until flush | 0 byte CPU copy in model | access/dirty | msync/flush 後才有指定寫回承諾 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.7 mmap 與 read/write 路徑的相同與差異 （續）

表中固定冷 page cache、無 read-ahead、完整一頁且呼叫成功；disk→cache 的 I/O 與 cache↔user 的 CPU copy 分開。page fault 是 exception，不是使用者主動呼叫的 system call。`write()` 與 `mmap` store 都可能先讓 cache 變 dirty，仍需另行達到指定持久性契約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.8 Kernel memory allocation

- kernel 不能直接使用一般 demand-paging 假設；中斷與 I/O 路徑可能需要非阻塞、實體連續或特殊生命週期的記憶體。
- slab allocator 依固定大小物件快取，降低配置與初始化成本；實作需處理 NUMA locality 與回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.8　Buddy system：以 21 KiB 請求重建配置

固定可用連續區域256 KiB，最小配置單位為4 KiB；下表位址以 KiB 為單位、使用左閉右開區間。buddy allocator 將非 power-of-two 請求上取整；21 KiB→32 KiB：

| step | available split | 配置結果 |
|---|---|---|
| 0 | `[0,256)` free | 尚未配置 |
| 1 | `[0,256)`→`[0,128)`,`[128,256)` | 兩個128 KiB buddies |
| 2 | `[0,128)`→`[0,64)`,`[64,128)` | 兩個64 KiB buddies |
| 3 | `[0,64)`→`[0,32)`,`[32,64)` | 兩個32 KiB buddies |
| 4 | 配置 `[0,32)` | 21 KiB request 使用32 KiB；內部浪費11 KiB |

此時未配置的 buddies 是 `[32,64)`、`[64,128)`、`[128,256)`，合計32+64+128=224 KiB。free bytes 足夠，但分屬不同 power-of-two blocks；若下一個請求需要64 KiB，可合併 `[0,64)` 上的兩個 buddies（前提是已配置的32 KiB先釋放），否則仍需另找適合區域。



## 9.8　Buddy system：以 21 KiB 請求重建配置 （續）

allocator invariant 是每個免費區塊屬於唯一的 block size 與 buddy pair，配置範圍不重疊；release 後只在 buddy 也 free 時合併。只把 free bytes 相加無法判斷請求能否滿足。

若區域起點對齊256 KiB，大小 $2^k$ 的 block（此處以 bytes 計）位於對齊位址 $a$，buddy 位址為 $a \operatorname{xor} 2^k$。釋放 `[0,32)` 後，先與 `[32,64)` 合成64，再與 `[64,128)` 合成128，最後與 `[128,256)` 合成256；每一步只查同 order 的那個 buddy。樹高 $\log_2(256/4) = 6$，分裂/合併最壞走6層，並非任意遍歷整個 free list。

## 9.8　Slab：物件重用與 fragmentation 的分母

固定 objects=256 bytes、slab page=4096 bytes，若完全不計 slab metadata，一個 page 可放 `4096/256=16` objects。實際 slab 會保留 page/slab 管理資訊，因此不能把16宣稱成所有實作的精確上限。

模型配置2個 objects、釋放1個：

| 指標 | 數值 | 意義 |
|---|---|---|
| live objects | 1 | 仍由 kernel 持有 |
| free objects | 15 | 可立即重用 |
| live bytes | 256 | 不包含 cache/slab metadata |
| slab page bytes | 4096 | 已留在 cache 的 frame |
| process RSS | 不由這個 object count 推得 | 分母是 process mapping |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.8　Slab：物件重用與 fragmentation 的分母 （續）

slab cache 把空物件留在 cache，能省 allocation、初始化與 page allocation；但 footprint 不會隨 live objects 立即下降。NUMA 系統還要保留 node locality，remote reclaim 與跨 node allocation 的延遲必須放在成本模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.9　其他考量

- page size 影響 mapping 粒度、TLB coverage、fault 頻率與內部碎裂。program prepaging 可載入可能需要的頁，但猜錯會增加 I/O。
- I/O interlock 與 pinned page 必須避免在裝置仍使用時換出；鎖定頁會降低可替換資源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.9　Page size、prepaging 與 pinning

對1 MiB virtual range：

| page size | page/PTE 數量 | TLB coverage | 可能浪費 |
|---|---|---|---|
| 4 KiB | 256 | 一個 TLB entry 只覆蓋4 KiB | 每物件尾部較小 |
| 64 KiB | 16 | 同樣 entries 可覆蓋16倍範圍 | 每物件尾部與 read amplification擴大 |

大 page 降低 walk/TLB footprint 與 fault count，但增加單頁 I/O 與 internal fragmentation；若應用有大量小物件，尾端浪費與 copy 量可能抵消 TLB 收益。

prepaging 在第一次使用前載入預期頁。固定 workload 有8 pages、locality{P1,P2,P3}，prepage三者成本3 reads，之後前三次 accesses0 faults；pure demand paging也做3 reads但三次 faults。兩者 data I/O 相同，差異在 trap、latency 與猜錯風險；多載入的P4若永不使用就是額外成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.9　I/O interlock 與 pinned page

裝置 DMA 使用 frame F40 時，F40 的 page count／pin state 必須阻止 reclaim。事件表：

| 時刻 | F40 owner | pin/DMA | reclaimable |
|---|---|---|---|
| t0 | user buffer | pin=0 | 是 |
| t1 | device queue | pin=1, DMA pending | 否 |
| t2 | device完成 | DMA=0，pin尚未清 | 否 |
| t3 | completion handler清 pin | pin=0 | 回到 LRU，才可選為 victim |

若 t1 錯把 F40 reclaim，device completion 會寫入已被其他 page 使用的 frame。t2 即使 DMA 已完成也不能在 completion protocol 未完成前假設 ownership 已歸還；pin 必須有明確 release/error path。

lock bit 與 mlock 是不同層次的例子，實際語意由 API 定義；共同不變量是「仍在使用的 frame 不得被重複配置」。長期限 pin 會減少 evictable pool，單看 free count 可能高估可用資源。



## 9.9　程式存取次序如何改變page locality

固定C/C++ row-major `int A[4][1024]`，int=4bytes、起點4KiB對齊、page=4KiB，每個row剛好一頁。只計此array的page references，2個frames、精確LRU、初始皆空；code/stack與cache-line成本另計。

| 走訪順序 | page reference型態 | fault推導 | 總faults |
|---|---|---|---|
| row外、column內 | 0重複1024次，再1、2、3各1024次 | 每row第一次fault，同行其餘1023次hit | 4 |
| column外、row內 | 0,1,2,3重複1024輪 | 兩frames保不住四頁；每頁再次使用前已被LRU淘汰 | 4096 |

column走訪前8次狀態（從最舊到最新）為 `[0]F,[0,1]F,[1,2]F,[2,3]F,[3,0]F,[0,1]F,[1,2]F,[2,3]F`，之後週期重複。相同16KiB資料與相同運算次數，存取排列便可改變page fault數；這不是硬體上每種loop都必然相差同一倍數，因真實OS可能分配更多frames、預取或改用其他政策。

## 9.10　作業系統例子

- OSC9e的Windows歷史模型以working-set min/max與free-list threshold決定配置、local replacement與trimming；本節給定A/B/C實際駐留數與free變化。
- 教材Solaris的front/back hands建立時間視窗，interval=handspread/scanrate；Linux v3.2另按PTE references、page類型、reclaim context與mapping/refcount決策，三者不共用一個門檻。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.10　Linux 3.2 reclaim 的觸發、掃描與逐頁處置

本頁固定 Linux v3.2、order-0 的非同步 reclaim 路徑。allocation slow path 在 zone 的 free pages 低於 low watermark 時可喚醒 `kswapd`；回收目標是恢復 watermark 所要求的可用頁，而不是「只要 free<low，每次掃描都必定成功」。`kswapd` 可掃 inactive anon/file lists，並依 active/inactive 平衡進行降級。`page_check_references()` 使用 PTE reference 與 PageReferenced 兩類狀態，`shrink_page_list()` 再依 unmap、dirty、writeback、mapping/refcount 決定結果。這是 Linux 3.2，沒有現代 multi-gen LRU。

固定本輪為 `current_is_kswapd=true`、`priority=9`、`DEF_PRIORITY=12`、`may_writepage=true`、允許進入檔案系統與必要的 I/O/unmap；page lock 成功，無並行重新引用或修改，P2/P3 的 mapping/refcount 可合法移除，P3 的 pageout 成功啟動非同步 I/O。這些條件使下表確實走到所列分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.10 Linux 3.2 reclaim 的觸發、掃描與逐頁處置 (續)

固定掃描輸入是五個 inactive pages：

| page | type | PTE referenced | dirty | writeback in progress | mlocked |
|---|---|---|---|---|---|
| P1 | anonymous | 1 | 0 | 0 | 0 |
| P2 | file | 0 | 0 | 0 | 0 |
| P3 | file | 0 | 1 | 0 | 0 |
| P4 | anonymous | 0 | 0 | 0 | 1 |
| P5 | file | 0 | 1 | 1 | 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 9.10 Linux 3.2 reclaim 的觸發、掃描與逐頁處置 (續)




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 9.10　Linux 3.2 reclaim 的觸發、掃描與逐頁處置 (續)

同一輪掃描的逐步決策如下：

| step | candidate | 判斷 | action | free-frame 增量 |
|---|---|---|---|---|
| 1 | P1 | referenced | activate 到 active LRU | 0 |
| 2 | P2 | clean、unreferenced、可移除 mapping | reclaim | +1 |
| 3 | P3 | dirty | 啟動 asynchronous writeback | 0 |
| 4 | P4 | mlocked | 移到 unevictable set | 0 |
| 5 | P5 | PageWriteback | defer | 0 |

只有 P2 在這一輪成為可重用 frame。P1 是 anonymous 頁且 PTE 確認被引用，故 activate；P3 已是選中的回收候選，但尚未回收，writeback 完成前不可重用；P4 因 mlock 進入 unevictable；P5 已在 I/O 中。單純額外的 DMA pin/refcount 不等同 mlock：它可能仍在一般 LRU，最終因 reference 無法移除而留下，不能一概畫成 unevictable。



## 9.10　後續狀態、版本界線與反例




下一輪掃描前，P3 的 writeback 可能仍在進行，也可能完成。完成只清除這次 writeback 的狀態，**不保證沒有並行的新寫入使 dirty 再度為1**。必須重新觀察 dirty=0、writeback=0、mapping/refcount 可移除，才可重用。P4 須合法解除 mlock/unevictable；等待時間不能自行解除鎖定。

## 9.10　後續狀態、版本界線與反例 （續）

固定狀態轉移可寫成：

| current event | required observable | next state | 不得做的事 |
|---|---|---|---|
| free below low watermark | kswapd wake / scan progress | scan | 未掃描就增加 free count |
| referenced | PTE/page reference | active | 當 unused 立即丟棄 |
| clean unreferenced | mapping/refcount removable | free | 忽略 pin |
| dirty | writeback submitted | writeback | 在 I/O 完成前重用 frame |
| writeback complete | 重新檢查 dirty=0、writeback=0，mapping/refcount 可移除 | 可回收 | 把 completion 當作永不再 dirty |
| mlocked | lock state cleared | evictable LRU | 因 waited long 而解 pin |



## 9.10　後續狀態、版本界線與反例 （續）

把 Solaris 的 `lotsfree/desfree/minfree`、Windows 的 working-set minimum/maximum 與 Linux watermarks 混成一個 threshold，會得到不存在的通用參數。本章保留這三個版本的機制名稱與責任邊界；數值只在明確標示的產品案例中出現。

上表的 activate/reclaim/writeback/exclude/defer 是一對一決策；此輪只有 P2 立即增加一個 free frame。教學模型省略 lock race 與實際 I/O 延遲，實作仍必須由核心狀態與 completion evidence 驗證。

## 9.10　改一個條件，為何回收結果就不同

| 改動 | Linux v3.2 分支 | 結果 |
|---|---|---|
| P1 改為 file 頁，只有首次 PTE reference，原 PageReferenced=0 | 設 PageReferenced，回傳 PAGEREF_KEEP | 留在 inactive，並非無條件 activate |
| P3 的 priority 改為10 | `priority >= DEF_PRIORITY-2` | 標記 reclaim 後保留，由其他 writeback 路徑處理 |
| P3 改為 direct reclaim、不是 kswapd | file-cache dirty 限制 | 本分支不直接寫 file page |
| P3 的 may_writepage=false | 禁止 pageout | 保留 dirty 頁，不能增加 free count |
| P2 有額外 pin/reference | 最後的 mapping/reference 移除失敗 | 保留，並非移到 unevictable 的同義詞 |
| P5 writeback 完成但又被寫 dirty | 重新遇到 dirty 分支 | 還不能當 clean frame 重用 |



## 9.10　改一個條件，為何回收結果就不同 （續）

因此不能用單一 `if referenced then activate; if dirty then writeback` 冒充此版本完整規則。reference 的來源、page 類型、scan context 和最後的所有權檢查都會改變結果。

## 9.10　教材 Windows working-set trimming 的操作例

採 OSC 9e §9.10.1 所述的歷史 Windows working-set 模型，與第19章 Windows 7 案例對讀；這是該書的簡化政策，非現行 Windows 所有修補版本的實作規格。working-set minimum/maximum 控制駐留頁範圍，超過 maximum 的細節由實作決定；free list 不足時，automatic trimming 可移除超過 minimum 的頁。

固定示範參數 threshold=3 frames、目前 free=1，三個 processes A/B/C 的 `(minimum,resident,maximum)` 分別為 `(2,4,4)`、`(2,3,4)`、`(2,2,4)`。本例待釋放頁全 clean、未鎖定，選 A→B→C，恢復到 free>3 停止。

| 動作 | A resident | B resident | C resident | free | 原因 |
|---|---|---|---|---|---|
| 初始 | 4 | 3 | 2 | 1 | 低於 threshold |
| A trim 兩頁 | 2 | 3 | 2 | 3 | A 超過 minimum 的2頁可移除 |
| B trim 一頁 | 2 | 2 | 2 | 4 | free 恢復至 threshold 以上 |
| C 不動 | 2 | 2 | 2 | 4 | 已在 minimum；不因本輪 trim 再減 |



## 9.10　教材 Windows working-set trimming 的操作例 （續）

後續 A fault 時，若尚在 maximum 以下且 free pool 足夠，可取得一頁；若在 maximum 且模型要求 local replacement，就在 A 的 working set 內選 victim。clustered demand paging 還可能連同鄰頁載入，所以「fault 一次恰增加一 resident page」只屬先前單頁模型。

## 9.10　教材 Solaris two-hand clock 的時間視窗

採 OSC 9e §9.10.2 的歷史 Solaris pageout 快照。free<lotsfree 開始 scanner，壓力加大時 scanrate 從 slowscan 提高；desfree/minfree 對應更積極回收或換出，不能把三者改成 Linux 同一個 low watermark。

front hand 清 R bit，back hand 延後檢查。若兩手相距 $h$ pages、穩定掃描率 $s$ pages/s，兩次觀察間隔為 $h/s$ 秒。固定示範 $h = 4, s = 100$，為0.04秒；此數值是例題參數，不是產品預設值。

| page | front hand 經過 | 中間事件 | back hand 看到 | 處置 |
|---|---|---|---|---|
| Q1 | R←0 | 再次讀取，R←1 | R=1 | 保留 locality |
| Q2 | R←0 | 沒再用，clean | R=0、clean | 解除映射後可回收/放 cache list |
| Q3 | R←0 | 沒再用，dirty | R=0、dirty | 寫回完成前不可重用 frame |

把 $s$ 提高到400而保持 (h=4)，間隔縮為0.01秒。同一頁更容易在短視窗內沒有再引用而被選中；壓力與 locality 的判斷會隨 scanner 速度一起改變。cache list 中尚未覆寫的乾淨頁可被重新取回，free list 的無效內容則不能當作仍保留原資料。



## 合法缺頁與非法存取的共同判斷順序

| 判斷 | 必須確認 | 不能直接當成什麼 |
|---|---|---|
| virtual range／mapping | 此位址是否有合法映射與backing | 所有not-present都是合法page-in |
| 權限／COW | requested read/write是否允許；COW是否適用 | 禁止write就無條件配新frame |
| resident／PTE | 已有frame、standby或需storage取入 | 每個fault都固定一次disk I/O |
| victim／writeback | frame可否解除映射、dirty是否已安全處理 | dirty／pinned也可直接重用 |
| 恢復執行 | 內容、mapping及權限一致後重試 | 同一page以後不會再被淘汰／refault |

同一physical frame可以被多個PTE共享；frame容量按不同physical frames計，不能把各process的resident-mapping數直接相加當成frame用量。配置／淘汰及COW都要追到最後reference與TLB／I/O使用者。

## 用同一成本模型比較fault，不用單一比例替代效能

固定本章12次reference、3frames、normal hit100ns，每fault總服務8ms（已含該次存取）。

| policy | faults | hits | 總成本 |
|---|---|---|---|
| FIFO | 9 | 3 | 72ms＋300ns |
| LRU | 10 | 2 | 80ms＋200ns |
| OPT | 7 | 5 | 56ms＋500ns |
| 本章CLOCK | 9 | 3 | 72ms＋300ns |

OPT使用future作比較下界；線上replacement仍要符合各自history規則。若fault服務時間或預取／clustering策略不同，不能只按fault個數套此表。

working set、frame allocation、PFF與scan壓力共同決定是否有足夠駐留資源。評估時同時追fault服務成本、有效工作／吞吐及等待，避免fault減少卻因其他瓶頸更慢。



## 參考資料

- *Operating System Concepts*, 9th Edition, Chapter 9, 印刷頁397–461.
  - 9.1 Background，printed pages 397-401：virtual memory 與 logical/physical separation。
  - 9.2 Demand Paging，printed pages 401-408：lazy loading、valid-invalid state、fault service 與 effective access time。
  - 9.3 Copy-on-Write，printed pages 408-409：fork page sharing、write fault 與 zero-fill-on-demand。
  - 9.4 Page Replacement，printed pages 409-421：FIFO、OPT、LRU、reference bits 與 second chance。
  - 9.5 Allocation of Frames，printed pages 421-425：minimum frames、equal/proportional allocation 與 local/global replacement。
  - 9.6 Thrashing，printed pages 425-430：working-set model 與 page-fault frequency。

## 參考資料 （續）

- *Operating System Concepts*, 9th Edition, Chapter 9, 印刷頁397–461.
  - 9.7 Memory-Mapped Files，printed pages 430-436：file page mapping、shared/private 與 writeback。
  - 9.8 Allocating Kernel Memory，printed pages 436-439：buddy system 與 slab allocation。
  - 9.9 Other Considerations，printed pages 439-445：prepaging、page size、TLB、I/O interlock 與 pin/lock。
  - 9.10 Operating-System Examples，printed pages 445-448：Windows working set 與 Solaris pageout/reclaim。
  - 9.11 Summary，printed page 448。
- Linux v3.2 `mm/vmscan.c`：`page_check_references`、dirty/writeback、`PageUnevictable`、reclaim activation 與 defer 條件。本章固定為 Linux 3.2，不推成現代 multi-gen LRU。
- Linux v3.2 `mm/memory.c`：fault 與 page-table/mapping path 的實作對照。



## 參考資料 （續）

本章的 3-frame replacement trace、working-set 窗口、PFF threshold、fault service 時間與五頁 reclaim 狀態均為明確輸入；它們不是實機 benchmark，也不能互相替代一般正確性證明。

- Linux mmap(2)：shared/private mapping、外部檔案變更與private可見性界線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study